

Feature Store Schema Definitions, Feature Registry, and Data Flow

Application base, entity-based feature split, and monitoring scorecard design

Purpose. This document consolidates the schema definitions discussed for a lending monitoring scorecard design. It includes the application base layer, feature tables, a feature registry design, an entity-based split of features, and a simple end-to-end data flow between the tables.

1. Design Overview

- Keep the application base as the full population, including included and excluded applicants, with explicit inclusion and exclusion flags.
- Create origination features separately from performance features so that point-in-time logic stays clear and reusable.
- Generate contract features only for eligible applicants or customers that pass the exclusion rules.
- Roll contract features up to application plus applicant level for the final monitoring scorecard view.
- Store feature definitions and governance metadata in a feature registry rather than embedding feature meaning only in code or column names.

2. Data Flow Diagram

The diagram below shows the recommended logical flow between the base table, eligibility layer, feature tables, registry, and final monitoring view.

```
[application_base]
|-- full population, flags, exclusion reasons
| \
|  \--> [application_snapshot_features] -----\
|
+----> [vw_eligible_applicants_for_contract_features] |
|
v
[eligible_application_contract_map]
|
v
[contract_12m_performance_features]
|
v
[customer_application_12m_performance_features] -----+-->
[vw_monitoring_scorecard_customer]

[feature_registry.feature_set] -----\
[feature_registry.feature_definition] ---- +--> governs all feature tables and the final
monitoring view
[feature_registry.feature_lineage] -----/
```

3. Feature Registry

The feature registry is the control plane of the feature store. It stores what each feature means, where it is stored, how it is calculated, who owns it, and what version of the logic is currently active. The registry should be maintained separately from the feature value tables.

3.1 `feature_registry.feature_set`

Purpose: one row per feature table or reusable feature bundle.

Field	Purpose
feature_set_id	Stable identifier for the feature set
feature_set_name	Business-friendly name of the feature set
entity_name	Main entity such as application, applicant, customer, contract
grain_description	For example one row per application_id plus applicant_id
storage_table	Physical table or view that stores the feature values
domain_name	Origination, behavioural, monitoring, LGD, EAD, and so on
refresh_frequency	Daily, monthly, event-driven, or ad hoc
point_in_time_join_supported	Whether historical joins are supported safely
training_enabled	Whether the feature set is approved for modelling
serving_enabled	Whether the feature set is approved for operational use
status	Draft, active, deprecated, retired
version	Version of the feature set release

3.2 `feature_registry.feature_definition`

Purpose: one row per feature column. This is the main place where business meaning and calculation logic are documented.

Field	Purpose
feature_id	Stable identifier for the feature
feature_set_id	Links the feature to its parent feature set
feature_name	Human-readable feature name
column_name	Physical column name in the table
data_type	Declared type of the feature
description	Short business description
business_definition	Precise rule for what the feature means
calculation_logic	High-level description or code reference
aggregation_window	Example: 3m, 12m, lifetime, at_application
source_tables	Upstream input tables
default_value	Optional default or fallback value
valid_from / valid_to	Dates when the logic is valid
feature_version	Version of the feature logic
owner_team	Owning squad or function
quality_rule_ref	Reference to quality or reconciliation checks
status	Draft, active, deprecated, retired

3.3 feature_registry.feature_lineage

Purpose: lineage and operational traceability for how the feature values were produced.

Field	Purpose
lineage_id	Stable lineage record identifier
feature_id	Feature linked to the lineage record
upstream_system	Source platform or system of record
upstream_object	Source table, file, or API object
transformation_stage	Raw, curated, feature, serving
transformation_ref	Notebook, SQL, job name, or code reference
pipeline_name	Orchestration job or pipeline name
pipeline_run_id	Specific execution instance
created_at	Timestamp of lineage capture

4. Entity-Based Split of Features

Features should be split by the entity they describe and by the observation point at which they are valid. For this monitoring design, the cleanest split is shown below.

Entity table	Grain	Main feature groups
application_snapshot_features	application_id + applicant_id	Origination and application-time features such as requested amount, income, bureau score, scorecard outputs, product, and channel
contract_12m_performance_features	application_id + applicant_id + contract_id	Contract-level 12-month delinquency, payment, balance, utilisation, and status features
customer_application_12m_performance_features	application_id + applicant_id	Rolled-up 12-month performance features across contracts for monitoring
vw_monitoring_scorecard_customer	application_id + applicant_id	Consumer-facing scorecard view combining origination and 12-month performance

Simple rule for placing a feature:

- If it was known at application time, put it in application_snapshot_features.
- If it is naturally measured at contract or account level over the observation window, put it in contract_12m_performance_features.
- If it is the roll-up used directly by the scorecard or monitoring consumer, put it in customer_application_12m_performance_features or the final monitoring view.

5. Schema Definition: application_base

Purpose: canonical base population for origination data, inclusion or exclusion tracking, reconciliation, and downstream feature generation. This table should contain both included and excluded applicants or customers, with flags to show downstream eligibility.

Field group	Example fields
Keys	application_id, applicant_id, customer_id
Application dates	application_date, application_timestamp, decision_date
Application attributes	product_type, channel_code, requested_amount, approved_amount, term_months, interest_rate_at_origination, repayment_type, loan_purpose
Applicant profile	age_at_application, employment_status_at_application, residential_status_at_application, marital_status_at_application, dependants_count_at_application
Income and affordability	declared_income_monthly, verified_income_monthly, declared_expense_monthly, net_disposable_income_monthly, debt_to_income_ratio_at_application, payment_to_income_ratio_at_application, loan_to_income_ratio_at_application
Bureau and external credit	bureau_score_at_application, active_credit_count_at_application, total_external_balance_at_application, ccj_count_at_application, default_count_at_application, worst_status_12m_at_application, worst_status_24m_at_application
Decision fields	application_scorecard_score, application_score_band, origination_pd, decision_code
Eligibility controls	is_excluded_flag, exclusion_reason_code, exclusion_reason_desc, is_included_for_contract_features_flag, is_included_for_monitoring_flag
Operational metadata	source_system, pipeline_run_id, created_at, updated_at

6. Schema Definition: vw_eligible_applicants_for_contract_features

Purpose: filtered subset of application_base used to generate contract and performance features. Grain: same as application_base but only for included population.

```
SELECT *  
FROM application_base
```

WHERE is_included_for_contract_features_flag = true;

7. Schema Definition: application_snapshot_features

Purpose: reusable origination feature table containing only application-time features. This is the clean feature layer derived from application_base.

Field group	Example fields
Keys and anchor	application_id, applicant_id, customer_id, application_date, feature_timestamp
Origination product features	product_type, channel_code, requested_amount, approved_amount, term_months, interest_rate_at_origination, repayment_type, loan_purpose
Applicant profile features	age_at_application, employment_status_at_application, residential_status_at_application, marital_status_at_application, dependants_count_at_application
Affordability features	declared_income_monthly, verified_income_monthly, declared_expense_monthly, net_disposable_income_monthly, debt_to_income_ratio_at_application, payment_to_income_ratio_at_application, loan_to_income_ratio_at_application
Bureau features	bureau_score_at_application, active_credit_count_at_application, total_external_balance_at_application, ccj_count_at_application, default_count_at_application, worst_status_12m_at_application, worst_status_24m_at_application
Scorecard outputs	application_scorecard_score, application_score_band, origination_pd, decision_code
Metadata	feature_version, source_system, pipeline_run_id, created_at

8. Schema Definition: eligible_application_contract_map

Purpose: maps included applicants or customers to contracts that are eligible for contract feature generation.

Field group	Example fields
Keys	application_id, applicant_id, customer_id, contract_id
Dates	application_date, contract_start_date,

	contract_end_date
Metadata	source_system, pipeline_run_id, created_at

9. Schema Definition: contract_12m_performance_features

Purpose: contract-level aggregated performance features over the 12-month observation window, generated only for eligible applicants or customers.

Field group	Example fields
Keys and window	application_id, applicant_id, customer_id, contract_id, observation_start_date, observation_end_date, feature_timestamp
Delinquency features	max_contract_dpd_12m, avg_contract_dpd_12m, days_contract_dpd_gt_0_12m, days_contract_dpd_ge_30_12m, days_contract_dpd_ge_60_12m, days_contract_dpd_ge_90_12m
Month-normalized delinquency	months_contract_dpd_gt_0_12m, months_contract_dpd_ge_30_12m, months_contract_dpd_ge_60_12m, months_contract_dpd_ge_90_12m
Ever and consecutive flags	ever_contract_30dpd_12m_flag, ever_contract_60dpd_12m_flag, ever_contract_90dpd_12m_flag, two_consecutive_months_contract_arrears_12m_flag, max_consecutive_months_contract_arrears_12m
Payment features	total_contract_payment_due_12m, total_contract_payment_made_12m, contract_payment_ratio_12m_avg, months_contract_underpaid_12m, months_contract_missed_payment_12m, max_contract_payment_shortfall_12m
Balance and utilisation	avg_contract_balance_12m, max_contract_balance_12m, min_contract_balance_12m, end_contract_balance_12m, avg_contract_limit_12m, avg_contract_utilisation_ratio_12m, max_contract_utilisation_ratio_12m
Status flags	forbearance_12m_flag, restructure_12m_flag, default_12m_flag
Metadata	feature_version, source_system, pipeline_run_id, created_at

10. Schema Definition: customer_application_12m_performance_features

Purpose: rolled-up performance feature table at the monitoring grain. This table aggregates contract-level performance to application plus applicant level.

Field group	Example fields
Keys and window	application_id, applicant_id, customer_id, observation_start_date, observation_end_date, feature_timestamp
Coverage	contract_count_12m, months_observed_count, days_observed_count
Rolled-up delinquency	max_dpd_12m, avg_dpd_12m, days_with_dpd_gt_0_12m, days_with_dpd_ge_30_12m, days_with_dpd_ge_60_12m, days_with_dpd_ge_90_12m
Month-normalized delinquency	months_with_dpd_gt_0_12m, months_with_dpd_ge_30_12m, months_with_dpd_ge_60_12m, months_with_dpd_ge_90_12m
Ever and consecutive flags	ever_30dpd_12m_flag, ever_60dpd_12m_flag, ever_90dpd_12m_flag, two_consecutive_months_dpd_gt_0_12m_flag, two_consecutive_months_dpd_ge_30_12m_flag, max_consecutive_months_in_arrears_12m
Payment features	total_payment_due_12m, total_payment_made_12m, payment_ratio_12m_avg, months_underpaid_12m, months_missed_payment_12m, max_payment_shortfall_12m
Balance and utilisation	avg_balance_12m, max_balance_12m, min_balance_12m, end_balance_12m, avg_utilisation_ratio_12m, max_utilisation_ratio_12m
Status flags	forbearance_12m_flag, restructure_12m_flag, default_12m_flag
Metadata	feature_version, source_system, pipeline_run_id, created_at

11. Schema Definition: vw_monitoring_scorecard_customer

Purpose: consumer-facing monitoring scorecard view that combines application-time features with 12-month performance features.

Field block	Contents
Keys	application_id, applicant_id, customer_id
Origination anchor	application_date
Origination features	Selected columns from application_snapshot_features such as product, requested amount, term, income, bureau score,

	and scorecard outputs
Performance features	Selected columns from customer_application_12m_performance_features such as max_dpd_12m, months_with_dpd_ge_30_12m, payment_ratio_12m_avg, avg_balance_12m, and default_12m_flag
Observation metadata	observation_start_date, observation_end_date

12. Field Placement Summary

Place the field in	When to use it
application_base	Raw or near-raw origination fields, inclusion flags, exclusion reasons, and operational lineage
application_snapshot_features	Derived origination features known at application time
contract_12m_performance_features	Contract-level observed performance over the 12-month window
customer_application_12m_performance_features	Roll-ups used directly for customer or applicant level monitoring
feature_registry tables	Feature definitions, ownership, lineage, and version control metadata
vw_monitoring_scorecard_customer	Consumer-ready combined scorecard fields

13. Versioning and Schema Evolution Notes

- Additive changes are preferred: add new feature columns and register them in the feature registry before activating them.
- Use the registry to maintain feature meaning, owner, valid_from date, and feature_version.
- Use data table metadata such as pipeline_run_id and created_at for operational traceability.
- Do not silently change the meaning of an existing feature column; create a new feature version instead.